

Self-Referential Relationship in Entity Framework Core

In this article, I will discuss **How to Configure Self-Referential Relationships in Entity Framework Core (EF Core)** with Examples. Please read our previous article discussing [How to Configure Many-to-Many Relationships in EF Core using Fluent API](#).

What is a Self-Referential Relationship in Entity Framework Core?

A Self-Referential Relationship (also known as a recursive relationship) is a relationship where an entity has a navigation property to another instance of the same entity type, i.e., when an entity references itself in a relationship. In other words, a table (or entity) has a foreign key that references its primary key. This is useful for representing hierarchical data or relationships within the same entity. Self-referential relationships are necessary to represent hierarchical data within a single table. Some scenarios include:

- An Employee entity can have a manager, and that manager is also an employee.
- A Category entity where each category can have a parent category.

Implementing Self-Referential Relationships in EF Core

I will show you how to Implement One-to-Many relationships using the following three approaches:

- Default Conventions
- Data Annotations
- Fluent API

Default Conventions for One to Many:

Let us first create the Employee entity to represent the One-to-Many Self-Referential Relationships in EF Core with Default Conventions. We want to define a One-to-Many relationship where an employee can have multiple subordinates, and each subordinate has exactly one manager. So, please create a class file named **Employee.cs** within the Entities folder and then copy and paste the following code.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Employee
    {
        public int EmployeeId { get; set; }
        public string Name { get; set; }
        // Self-Referential Relationship
        public int? ManagerId { get; set; }
        public Employee Manager { get; set; }
        public ICollection<Employee> Subordinates { get; set; } = new
List<Employee>();
    }
}
```

Explanation:

- The ManagerId property acts as the foreign key.
- The Manager property is a reference to the employee's manager.
- The Subordinates property is a collection of employees that reports to this employee.

DbContext Configuration:

Next, modify the **EFCoreDbContext** class as follows:

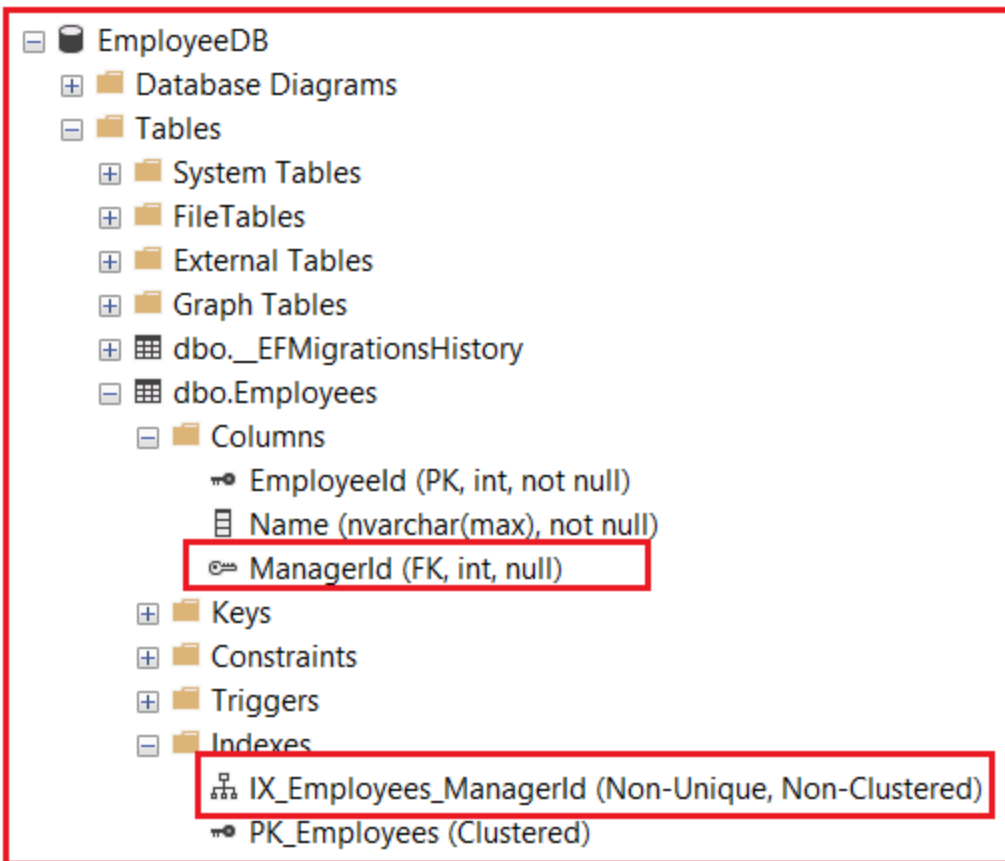
```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Configuring the Connection String
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=EmployeeDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        public DbSet<Employee> Employees { get; set; }
    }
}
```

Migrate and Update Database

Please execute the following command using the Package Manager Console:

- **Add-Migration InitialCreate**
- **Update-Database**

Once you execute the above commands, it should create the EmployeeDB with the Employees table with the following columns:



As you can see, the ManagerId column is created as a foreign key column pointing to the EmployeeId column of the same table. Further, you will notice that it is creating a Non-Unique and Non-Clustered Index on the ManagerId foreign key column, which is used for implementing one-to-many relationships.

Testing the One-to-Many Self-Referential Relationships Functionality:

Let us test the One-to-Many Self-Referential Relationships Functionality in EF Core by adding a few employees and then displaying the employees information in a hierarchical order.

```
using Microsoft.EntityFrameworkCore;
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            try
            {
                // Initialize the database context
                using (var context = new EFCoreDbContext())
                {
                    // Create managers and their subordinates
                    InsertEmployees(context);
                    // Fetch and display the tree structure
                    DisplayEmployeesTree(context);
                }
            }
        }
    }
}
```

```

    }
    catch (DbUpdateException ex)
    {
        // Exception Database Exception
        Console.WriteLine($"Database Error:
{ex.InnerException?.Message ?? ex.Message}");
    }
    catch (Exception ex)
    {
        // Exception handling to catch any errors
        Console.WriteLine($"Error occurred: {ex.Message}");
    }
}
// Method to insert a manager and their subordinates
static void InsertEmployees(EFCoreDbContext context)
{
    // Check if the database already has employees
    if (context.Employees.Any())
    {
        Console.WriteLine("Employees already exist in the
database.\n");
        return;
    }
    // Create two manager employees
    var manager1 = new Employee { Name = "Alice Manager" };
    var manager2 = new Employee { Name = "Bob Manager" };
    // Create subordinates under manager1
    var subordinate1 = new Employee { Name = "Charlie Employee",
Manager = manager1 };
    var subordinate2 = new Employee { Name = "David Employee",
Manager = manager1 };
    // Create subordinates under manager2
    var subordinate3 = new Employee { Name = "Eve Employee", Manager
= manager2 };
    var subordinate4 = new Employee { Name = "Frank Employee",
Manager = manager2 };
    var subordinate5 = new Employee { Name = "Grace Employee",
Manager = manager2 };
    // Add managers and subordinates to the context
    context.Employees.AddRange(manager1, manager2, subordinate1,
subordinate2, subordinate3, subordinate4, subordinate5);
    // Save changes to the database
    context.SaveChanges();
    Console.WriteLine("Managers and their subordinates have been
successfully added to the database.\n");
}
// Method to display employees in a tree structure
static void DisplayEmployeesTree(EFCoreDbContext context)
{
    // Retrieve managers along with their subordinates from the
database

```

```

        var managers = context.Employees
            .Include(e => e.Subordinates) // Include subordinates in the
query
            .Where(e => e.ManagerId == null) // Only select employees who
are managers (no ManagerId)
            .ToList();
        // Display the managers and their subordinates
        foreach (var manager in managers)
        {
            Console.WriteLine($"Manager Id: {manager.EmployeeId}, Name:
{manager.Name}");
            foreach (var subordinate in manager.Subordinates)
            {
                Console.WriteLine($"    \tSubordinate: Id:
{subordinate.EmployeeId}, Name:{subordinate.Name}");
            }
            Console.WriteLine(); //Line Spacing for better viewability
        }
    }
}

```

Output:

```

Managers and their subordinates have been successfully added to the database.

Manager Id: 1, Name: Alice Manager
    Subordinate: Id:3, Name:Charlie Employee
    Subordinate: Id:4, Name:David Employee

Manager Id: 2, Name: Bob Manager
    Subordinate: Id:5, Name:Eve Employee
    Subordinate: Id:6, Name:Frank Employee
    Subordinate: Id:7, Name:Grace Employee

```

Data Annotation Approach to Implement Self-Referential Relationship

Data Annotations allow us to configure EF Core relationships directly within entity classes using attributes. This approach is straightforward and keeps configurations close to the data model. Let us see how to Implement a One-to-Many Self-Referential Relationship using EF Core Data Annotation Attributes. So, for this, please modify the Employee Entity as follows:

```

using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    public class Employee
    {
        public int EmployeeId { get; set; }
        public string Name { get; set; }
        [ForeignKey("Manager")]
        public int? ManagerId { get; set; }
    }
}

```

```

        public Employee Manager { get; set; }
        [InverseProperty("Manager")]
        public ICollection<Employee> Subordinates { get; set; } = new
List<Employee>();
    }
}

```

Explanation:

- The [ForeignKey("Manager")] Data annotation attribute explicitly defines the foreign key relationship.
- The [InverseProperty("Manager")] Data annotation attribute tells Entity Framework Core that the Subordinates collection is the inverse of the Manager property.

With the above changes in place, generate the Migration, Update the database, and test the functionality, and it should work as expected. Once you run the application, you should see the following output:

```

Employees already exist in the database.

Manager Id: 1, Name: Alice Manager
      Subordinate: Id:3, Name:Charlie Employee
      Subordinate: Id:4, Name:David Employee

Manager Id: 2, Name: Bob Manager
      Subordinate: Id:5, Name:Eve Employee
      Subordinate: Id:6, Name:Frank Employee
      Subordinate: Id:7, Name:Grace Employee

```

Fluent API Approach to Implement Self-Referential Relationship

The Fluent API provides a more flexible way to configure entity relationships. It is useful for complex configurations that are not easily handled by conventions or data annotations. Let us see how to implement a One-to-Many Self-Referential Relationship using the EF Core Fluent API. First, modify the Employee Entity as follows. Here, you can see we are removing the Data Annotation Attributes.

```

namespace EFCoreCodeFirstDemo.Entities
{
    public class Employee
    {
        public int EmployeeId { get; set; }
        public string Name { get; set; }
        public int? ManagerId { get; set; }
        public Employee Manager { get; set; }
        public ICollection<Employee> Subordinates { get; set; } = new
List<Employee>();
    }
}

```

Configure the Relationship Using Fluent API:

Next, we need to define the one-to-many relationship between Manager and Employee, and we need to do this by overriding the OnModelCreating method of the DbContext class. So, modify the EFCoreDbContext class as follows:

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Configuring the Connection String
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=EmployeeDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            modelBuilder.Entity<Employee>() //Refers to the Employee Entity
                .HasOne(e => e.Manager) //Each employee has one Manager
                .WithMany(m => m.Subordinates) //Each Manager can have multiple employees
                .HasForeignKey(e => e.ManagerId) //ManagerId is the Foreign Key
                .OnDelete(DeleteBehavior.Restrict);
        }
        public DbSet<Employee> Employees { get; set; }
    }
}
```

Explanation:

- **HasOne(e => e.Manager)** specifies that each employee has one manager.
- **WithMany(m => m.Subordinates)** defines that a manager can have many subordinates.
- **HasForeignKey(e => e.ManagerId)** defines the foreign key in the relationship.
- **OnDelete(DeleteBehavior.Restrict)** prevents the deletion of an employee if they are assigned as a manager to others.

With the above changes in place, generate the Migration, Update the database, and test the functionality, and it should work as expected. Once you run the application, you should see the following output:

```
Employees already exist in the database.

Manager Id: 1, Name: Alice Manager
Subordinate: Id:3, Name:Charlie Employee
Subordinate: Id:4, Name:David Employee

Manager Id: 2, Name: Bob Manager
Subordinate: Id:5, Name:Eve Employee
Subordinate: Id:6, Name:Frank Employee
Subordinate: Id:7, Name:Grace Employee
```

Another Real-time Application of Self-Referential Relationships in EF Core:

Let's build a real-time application to manage Categories and Products using Entity Framework Core (EF Core). In this application:

- Categories can have multiple levels (self-referential one-to-many relationships), e.g., Parent Category → Subcategory → Sub-Subcategory).
- Products are associated with Categories, where each Product belongs to exactly one Category.

Steps Overview:

- Define the Category and Product entities.
- Configure the One-to-Many Self-Referential Relationship for the Category entity.
- Define the relationship between Product and Category.
- Seed data to create a three-level category hierarchy.
- Write sample queries to fetch data for real-time use.

Category and Product Entities

Let us first create the Entities required for our application.

Category Entity:

Create a class file named **Category.cs** within the Entities folder, and then copy and paste the following code. Here, we are implementing One-to-Many Self-referential relationships between Category, Subcategory. Also, implementing One-to-many relationships between Category and Product, i.e., one category can have multiple products.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Category
    {
        public int CategoryId { get; set; }
        public string Name { get; set; }
        // One to Many Self-referential relationship (Parent and
        Subcategories)
        // Foreign Key
        public int? ParentCategoryId { get; set; }
        public Category ParentCategory { get; set; }
        public ICollection<Category> Subcategories { get; set; } = new
        List<Category>();
        // One to Many Relationship with Products
        public ICollection<Product> Products { get; set; } = new
        List<Product>();
    }
}
```

Product Entity:

Create a class file named **Product.cs** within the Entities folder and then copy and paste the following code.

```
namespace EFCoreCodeFirstDemo.Entities
{
```



```

public class Product
{
    public int ProductId { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
    // Each product belongs to exactly one Category
    // Foreign Key
    public int CategoryId { get; set; }
    // Navigation Property
    public Category Category { get; set; }
}
}

```

Creating the DbContext

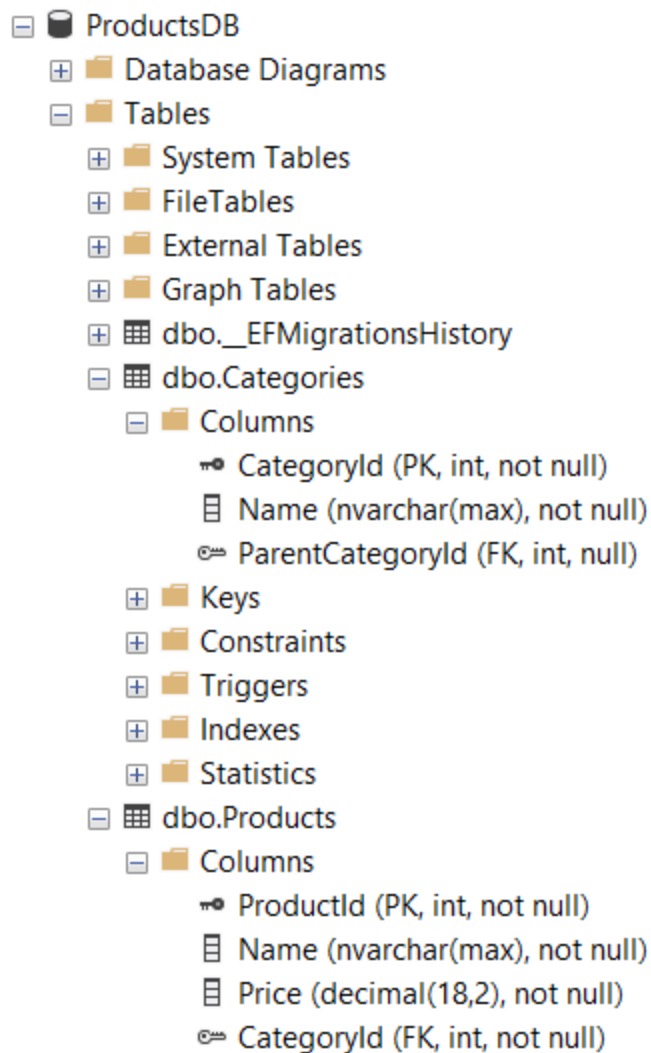
Modify the EFCoreDbContext class as follows:

```

using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Configuring the Connection String
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=ProductsDB;Trusted_Connection=True;TrustSer
        }
        public DbSet<Category> Categories { get; set; }
        public DbSet<Product> Products { get; set; }
    }
}

```

With the above changes, generate the Migration and Update the database using the Add-Migration and Update-Database commands in the Package Manager Console. Once you execute the commands, verify the database, and you should see the following:



Inserting Data with Three Levels of Categories:

Next, modify the Program class as follows. The following code is self-explained, so please read the comment lines for a better understanding.

```
using Microsoft.EntityFrameworkCore;
using EFCoreCodeFirstDemo.Entities;
using System;
using System.Linq;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            try
            {
                // Adding categories and products to the database
                AddCategories();
                // Displaying all categories, subcategories, and products in
                level 3 categories
            }
        }
    }
}
```

```

        DisplayCategories();
    }
    catch (DbUpdateException ex)
    {
        // Exception handling to catch database errors, showing the
inner exception if available
        Console.WriteLine($"Database Error:
{ex.InnerException?.Message ?? ex.Message}");
    }
    catch (Exception ex)
    {
        // Exception handling for any other errors
        Console.WriteLine($"An error occurred: {ex.Message}");
    }
}
// Method to add categories and products to the database
static void AddCategories()
{
    using var context = new EFCoreDbContext();
    // Check if the database already has products to avoid
duplication
    if (context.Products.Any())
    {
        Console.WriteLine("Products and Categories already exist in
the database.\n");
        return;
    }
    // Creating categories (Level 1 → Level 2 → Level 3)
    var electronics = new Category { Name = "Electronics" }; // Level
1
    var computers = new Category { Name = "Computers", ParentCategory
= electronics }; // Level 2
    var laptops = new Category { Name = "Laptops", ParentCategory =
computers }; // Level 3
    var phones = new Category { Name = "Phones", ParentCategory =
electronics }; // Level 2
    var smartPhones = new Category { Name = "Smartphones",
ParentCategory = phones }; // Level 3
    // Creating categories (Level 1 → Level 2 → Level 3)
    var homeAppliances = new Category { Name = "Home Appliances" };
// Level 1
    var kitchen = new Category { Name = "Kitchen", ParentCategory =
homeAppliances }; // Level 2
    var coffeeMachines = new Category { Name = "Coffee Machines",
ParentCategory = kitchen }; // Level 3
    // Creating products for each category
    var product1 = new Product { Name = "Dell Laptop", Price =
899.99M, Category = laptops };
    var product2 = new Product { Name = "HP Laptop", Price = 799.99M,
Category = laptops };

```

```

        var product3 = new Product { Name = "Espresso Machine", Price =
199.99M, Category = coffeeMachines };
        var product4 = new Product { Name = "iPhone 13", Price = 999.99M,
Category = smartPhones };
        var product5 = new Product { Name = "Samsung Galaxy S21", Price =
899.99M, Category = smartPhones };
        // Adding categories and products to the database
        context.Categories.AddRange(electronics, computers, laptops,
homeAppliances, kitchen, coffeeMachines, phones, smartPhones);
        context.Products.AddRange(product1, product2, product3, product4,
product5);
        context.SaveChanges();
        // Confirmation message after seeding data
        Console.WriteLine("Categories and Products added
successfully.\n");
    }
    // Method to fetch and display all categories, subcategories, and
products for level 3 categories
    static void DisplayCategories()
    {
        using var context = new EFCoreDbContext();
        // Fetch top-level categories (ParentCategoryId is null) and
include subcategories and products
        var categories = context.Categories
            .Include(c => c.Subcategories)
            .ThenInclude(c => c.Subcategories)
            .ThenInclude(c => c.Products) // Include products
for level 3 categories
            .Where(c => c.ParentCategoryId == null) // Fetch only top-
level categories
            .ToList();
        // Display all categories in a hierarchical format
        Console.WriteLine("All Categories, Subcategories, and Products
(in level 3 categories):");
        foreach (var category in categories)
        {
            Console.WriteLine($" \nCategory: {category.Name}"); // Level
1
            foreach (var subcategory in category.Subcategories)
            {
                Console.WriteLine($" Subcategory: {subcategory.Name}");
// Level 2
                foreach (var subSubcategory in subcategory.Subcategories)
                {
                    Console.WriteLine($" Sub-Subcategory:
{subSubcategory.Name}"); // Level 3
                    // Display products under each Level 3 category
                    if (subSubcategory.Products.Any())
                    {
                        Console.WriteLine(" Products:");
                        foreach (var product in subSubcategory.Products)

```

```

        {
            Console.WriteLine($"
{product.Name} ({product.Price})");
        }
    }
    else
    {
        Console.WriteLine("
this category.");
    }
}
}
}
Console.WriteLine(); // New line for better output formatting
}
}
}
}

```

You will get the following output when you run the above application code.

```

Categories and Products added successfully.

All Categories, Subcategories, and Products (in level 3 categories):

Category: Electronics
  Subcategory: Computers
    Sub-Subcategory: Laptops
      Products:
        - Dell Laptop ($899.99)
        - HP Laptop ($799.99)
  Subcategory: Phones
    Sub-Subcategory: Smartphones
      Products:
        - iPhone 13 ($999.99)
        - Samsung Galaxy S21 ($899.99)

Category: Home Appliances
  Subcategory: Kitchen
    Sub-Subcategory: Coffee Machines
      Products:
        - Espresso Machine ($199.99)

```

When Should We Use Self-Referential Relationships in Entity Framework Core?

We need to use self-referential relationships when we model hierarchical or recursive data within a single entity. Some of the common scenarios include:

- Organizational structures (employees reporting to managers).
- Hierarchical data (e.g., categories and subcategories).

These relationships are beneficial when the relationship structure stays within the same entity type. They can avoid additional tables or unnecessary complexity in the model.

In the next article, I will discuss [Global Configurations in Entity Framework Core using Fluent API](#) with Examples.
In this article, I explain **How to Configure Self-Referencing Relationships in Entity Framework Core** with Examples.
I hope you enjoyed this article.